Schema-Bound Tokens

Attestor Sets as Mint Authority on Attestation-Native Chains

Stefan Stefanović

Ligate Labs

Working Paper v0.2

Date: 2026-05-25

Contact: hello@ligate.io

Abstract

Most blockchains expose two token primitives at the runtime layer: fungible tokens (admin-mintable balances) and non-fungible tokens (admin-mintable unique items). The mint authority in both cases is a single address whose only on-chain accountability is its bonded stake, if any. A third primitive becomes natural on a chain that already runs attestor sets as a first-class object: bind the mint authority to an `AttestorSetId` rather than to a single address, and make every mint event itself an attestation under a canonical system schema.

This note specifies that primitive (schema-bound tokens) as a research object. The engineering design lives in ligate-chain#286. The contribution here is the formal-properties analysis: authorization equivalence (any actor who can produce a valid attestation under the bound schema can authorize a mint, and conversely), auditability (mint events are queryable as attestations under `chain.token-mint/v1`, not as opaque state diffs), composition with the per-schema fee market, and liveness under attestor-set turnover.

The differentiator from existing threshold-issuance patterns (multisig wallets, FROST-based protocols, EAS revocable attestations) is that Ligate's threshold verification is *native to the chain's attestation module*, not a separate contract or off-chain protocol. The reputation layer (Proof of Useful Attestation, v0.8) then provides an economic floor: bad-faith mints by the attestor set damage the same reputation that backs every other attestation they sign. This is the on-thesis token primitive for an attestation-native chain. The chain's distinguishing primitives carry the token's authorization, audit trail, and economic security in one piece.

---

1. Background

1.1 The token-primitive landscape on Ligate Chain

Per ligate-chain#286, v1 of Ligate Chain exposes four token primitives at the runtime layer:

1. **Standard fungible tokens** (ligate-chain#47). Mint authority: single address. Issuance auditability: state-diff only. Familiar ERC-20 shape.
2. **Standard non-fungible tokens** (ligate-chain#48). Mint authority: single address. Issuance auditability: state-diff only. Familiar ERC-721 shape.
3. **Schema-bound fungible tokens**. Mint authority: `AttestorSetId` (threshold quorum). Mint events: attestations under `chain.token-mint/v1`.
4. **Schema-bound non-fungible tokens**. Same authorization model as (3), distinct asset shape.

The first two primitives are commodity. They exist so partners building on Ligate can deploy familiar token shapes without learning a new model. The last two are the architecturally on-thesis primitives. They use the chain's distinguishing objects (schemas + attestor sets) as the foundation of the token's authorization model.

1.2 Why threshold mint authority is the right default

A single-address mint authority is a single point of compromise. Industry workarounds include multisig wallets and external threshold-signature protocols, but those are *bolted on* to chains that did not design for them. The chain sees the multisig contract's output, not the quorum it represents. The auditability of "who actually authorized this mint" lives outside the chain.

A chain whose runtime already runs attestor sets as first-class state objects can do better. The attestor set's threshold quorum is verified natively in the consensus pipeline. The mint event itself becomes an attestation, queryable under a system schema, with the same threshold-signature semantics as every other attestation on the chain. There is no separate ledger and no separate trust layer. The issuance log is part of the attestation log.

1.3 Position relative to PoUA

PoUA v0.8 establishes attestor sets as the primary trust primitive for application-layer correctness, with non-transferable reputation tied to the validators who include valid attestations. Schema-bound tokens reuse the same attestor-set object and inherit the same reputation feedback loop: if an attestor set authorizes a fraudulent mint, the validators who include that attestation are subject to the same reputation mechanics that govern any §A.3 grinding behavior. The reputation layer is not retrofitted to handle tokens; tokens are simply another application of the layered defense PoUA already specifies.

---

2. The schema-bound primitive: formal definition

2.1 Mint authority binding

Each schema-bound token type is uniquely identified by

```
token_id = SHA-256(domain_tag || mint_authority || name || decimals)
```

where `mint_authority = AttestorSet(A)` for some registered attestor set `A`. The token id is bound to the specific attestor set at construction time. Subsequent rotation of the attestor set (per Ligate Chain's attestor-set management; #286 §3.4) does not invalidate the token id.

2.2 Mint as attestation

A mint event that creates `n` units of token type `T` at recipient `r` is recorded as an attestation under the system schema `chain.token-mint/v1` (call it `sigma_mint`), with payload

```
p = (token_id(T), r, n, nonce, metadata)
```

signed by a threshold quorum of the attestor set `A` at the threshold `k` already on record for `A`. The attestation is *valid* if and only if it carries a `k`-of-`|A|` threshold signature over `(p, sigma_mint, submitter)`.

2.3 Recall semantics

An attestor set may optionally bind a token type with a `recall_by_authority` flag at construction. When set, the same attestor set that authorized minting may also authorize burns from arbitrary holder balances, with the same threshold semantics. Recall is itself an attestation under `chain.token-burn/v1`; the audit trail is symmetric.

2.4 Recoverability under attestor-set turnover

The token id binds to the attestor set's *registered identity* (`AttestorSetId`), not to its key material. When the attestor set rotates keys (replaces members or threshold), existing minted balances remain

valid (mint events are historical attestations; their threshold-signature verification uses the keys recorded at the time of mint). Future mints use the rotated key set. This decouples token-balance persistence from key-rotation events, the same way the chain decouples attestation persistence from attestor-set membership changes.

---

3. Formal properties

This is the priority section: what's actually provable about the primitive once §2 is fixed. Five formal claims, each stated and argued.

3.1 Authorization equivalence

Claim. Any actor that can produce a valid attestation under a schema σ whose attestor set is $\mathcal{A}$ can authorize a mint of any token type $\mathcal{T}$ whose `mint_authority` is `AttestorSet(A)`, and conversely.

Argument. Validity of both the schema attestation and the mint attestation reduces to the same predicate: a $k_{\mathcal{A}}$-of-$|\mathcal{A}|$ threshold signature over the canonical Borsh encoding of the payload (with the schema id as part of the signed bytes). The schema id differs (the application schema for one, σ_{mint} for the other), but the signing quorum is identical. Forward direction: a quorum that can produce a valid signature for schema σ can also produce a valid signature for σ_{mint} (the cryptographic primitive is the same). Reverse direction: a quorum that can produce a valid signature for σ_{mint} can also produce a valid signature for σ (same reasoning). The equivalence is symmetric. □

Consequence. Designers who decide to trust an attestor set with schema attestation work have also implicitly decided to trust them with mint authorization for any token bound to that same set. This is a feature, not a bug: it forces the same trust-modeling discipline at one decision point rather than two.

3.2 Auditability via the attestation log

Claim. For any token type $\mathcal{T}$ schema-bound to $\mathcal{A}$, the full set of mint events that contributed to any holder's balance is queryable as a finite set of attestations under σ_{mint} filtered by their `token_id` field matching `token_id(T)`.

Argument. Every mint is recorded as an attestation per §2.2. Attestations are stored in the chain's attestation log (per PoUA v0.8 §3.7 system diagram). The token's runtime state (current balances) is derivable from the cumulative sum of mint events minus burn events; both classes are attestations. There is no off-attestation-log state mutation for schema-bound tokens. □

Consequence. Audit infrastructure that already indexes attestations (chain explorers, partner indexers, downstream analytics) gets token-issuance audit for free. No separate issuance ledger needs to be tracked. Investigators reconstructing "who minted what when" use the same queries as investigators reconstructing "who attested what when."

3.3 Composition with the per-schema fee market

Claim. A mint event under σ_{mint} pays exactly the per-attestation fee for σ_{mint}, plus the standard chain gas for the transaction carrying it. The mint does not pay an additional, separate "token-mint

fee" on top of the attestation fee.

Argument and open question. The attestation module's per-schema fee market (per the Per-Schema Fees paper, v0.1.1) prices attestations by schema. σ_{mint} is a system schema; its fee is set by governance and not subject to the open-fee-market dynamics that apply to application schemas. The bound mint fee should be calibrated so that:

- Routine mint volume does not exhaust block space (a fee floor)
- Adversarial mint flooding (a quorum that controls $\mathcal{A}$ tries to spam mints to inflate $\mathcal{T}$'s supply) is economically deterred (a fee ceiling-equivalent)

Open question. Whether the mint fee should also include a per-unit-minted component (a Pigouvian tax on supply expansion, recoverable to the chain's τ_{burn} pool) is an economics RFC question, tracked at ligate-chain#258. Without it, the per-attestation fee is the only cost regardless of n, which may be too cheap for adversarial high-supply mints. Documented here; not resolved at v0.1.

3.4 Liveness under attestor-set turnover

Claim. Holders of $\mathcal{T}$ retain valid balances when the attestor set $\mathcal{A}$ rotates members or threshold, provided the rotation is performed through the chain's attestor-set management module (not via off-chain key compromise + re-registration).

Argument. Mint events are historical attestations. Their validity at the time of mint is determined by the threshold-signature verification under the keys recorded for $\mathcal{A}$ *at the slot of the mint attestation*. A rotation that changes $\mathcal{A}$'s key set at slot t updates the verifier state for attestations at slot $t' > t$ but does not retroactively invalidate attestations at slot $t' \leq t$. The historical attestation log is append-only by construction (per PoUA v0.8 §3.7); rotation events are recorded as their own attestations under the attestor-set management schema. Token balances derived from pre-rotation mints remain in the chain's runtime state regardless of post-rotation key set. □

Edge case. If $\mathcal{A}$ is *removed* entirely (governance action, end-of-life), the token type's `mint_authority` is permanently revoked; the runtime can no longer accept new mints (no valid signing quorum exists), but pre-removal balances remain valid. This is the intended end-of-life path for a sunset token, similar to a smart-contract owner renouncing admin in conventional patterns.

3.5 Reputation feedback loop

Claim. Bad-faith mints by $\mathcal{A}$ are subject to the same reputation mechanics that govern any §A.3-grinding behavior under PoUA. There is no separate "token issuance reputation" track.

Argument. Mint events are attestations under σ_{mint}, included by validators in blocks. Per PoUA v0.8 §4.3 the reputation update rewards validators who include valid attestations and exposes them to the standard slashing conditions if those attestations are invalid or detected as part of a grinding pattern (§A.2 / §A.3). If $\mathcal{A}$ issues a fraudulent mint (a mint exceeding a stated cap, or a mint contradicting an off-chain authoritative source), the chain has the same recourse it has for any fraudulent attestation: detect, slash the validator who included it, and trigger appeal via §5.5.5.

Limit of this argument. The chain detects *invalid* attestations (failed threshold signature) and *graph-shaped misbehavior* (§A.3 bipartite-density). It does not detect *semantically incorrect* mints (the attestor set issues a mint that satisfies cryptographic validity but is contractually unauthorized). Semantic correctness is the schema designer's problem, the same way it is for any application-layer

schema. The reputation feedback loop bounds the cost of provably-bad behavior, not the cost of debatable behavior.

3.6 Fee-market composition

Claim. Mint events under σ_{mint} price into the per-schema fee market specified in Per-Schema Fees v0.2 §4 with no special-casing: σ_{mint} has its own per-schema base fee $b_{\sigma_{\text{mint}}}$, its own target utilization $T_{\sigma_{\text{mint}}}$, and its own routing fraction $\rho_{\sigma_{\text{mint}}}$, all set at system-schema registration and adjustable via governance. The PoUA Lemma 1 cost-to-grind floor preserved in §5.1 of the Per-Schema Fees paper applies per-schema, so mint-event grinding against the token-supply ceiling pays the same floor as any other per-schema attestation grinding.

Argument. Mint events are attestations under σ_{mint}. Per-Schema Fees v0.2 §3.1 defines `FeeState($\sigma$)` per schema; σ_{mint} inherits this state like every other registered schema. The §4.1 base-fee adjustment formula applies: when mint volume exceeds $T_{\sigma_{\text{mint}}}$ relative to the schema's allocated block capacity, $b_{\sigma_{\text{mint}}}$ climbs. When mint volume is low, it falls. The §4.4 burn split applies: τ_{burn} of every paid mint fee is burned (preserving Lemma 1 floor); $(1 - \tau_{\text{burn}}) \cdot \rho_{\sigma_{\text{mint}}}$ flows to the system-treasury (the canonical recipient for system-schema routing per Per-Schema Fees v0.2 §4.4); $(1 - \tau_{\text{burn}}) \cdot (1 - \rho_{\sigma_{\text{mint}}})$ flows to the validator who included the mint. □

Calibration recommendation for σ_{mint}. Per the Per-Schema Fees v0.2 §4.2 demand-profile taxonomy, σ_{mint} matches the "low-volume, high-value" profile (mint events are rare but each carries economic significance equal to the token supply being expanded). Recommended starting values:

- $T_{\sigma_{\text{mint}}} = 0.7$ (matches the high-value profile recommendation; mint flow has predictable shape, can pack closer to demand peak)
- $\rho_{\sigma_{\text{mint}}} = 0.0$ (system schema; non-burned share goes entirely to validators rather than to a "schema author," since the chain itself is the author)
- $b^{\min}_{\sigma_{\text{mint}}}$, $b^{\max}_{\sigma_{\text{mint}}}$ governance-tunable; reasonable starting bounds at 10^3 and 10^9 uavow respectively (1 milliavow floor; 1000 AVOW ceiling per mint).

Effect on adversarial mint flooding. §3.3's open question on adversarial mint flooding by an attestor set is partially closed by §3.6. A quorum that controls $\mathcal{A}$ and submits many mints in quick succession drives $u_{\sigma_{\text{mint}}}$ above $T_{\sigma_{\text{mint}}}$, which climbs $b_{\sigma_{\text{mint}}}$ via §4.1. The adversary's per-mint cost rises as they flood. Combined with the §3.5 reputation feedback loop and the §4.1 attestor-set incentive analysis below, the fee market is the economic-deterrent dimension of the layered defense; reputation is the long-run-revenue dimension; chain detection is the structural dimension. Three layers, each independently breakable, conjoint hard.

Open question still unresolved. Whether the chain should add a separate per-unit-minted Pigouvian tax (in addition to the per-attestation fee) is tracked at ligate-chain#258. v0.2 of this note closes the per-attestation-fee composition (this §3.6); the per-unit-minted tax is a separate economic-design question.

---

4. Game-theoretic concerns and their resolution

Five items the v0.1 draft surfaced explicitly. v0.2 resolves three of them (1, 3, 5) using primitives that shipped in companion papers this cycle. Items 2 and 4 remain open and tagged for v0.3+ work.

4.1 Attestor-set incentive to issue beyond stated cap (resolved at v0.2)

The v0.1 framing left this open pending token-economic parameter specification. v0.2 closes it using the Per-Schema Fees v0.2 §6.4 incentive analysis as the analytical frame.

Setup. An attestor set $\mathcal{A}$ controls mint authority for token type $\mathcal{T}$ with stated supply cap $C_{\mathcal{T}}$. The marginal-revenue gain from issuing one excess unit beyond the cap is $r_{\mathcal{T}}$ (the unit's market price). The marginal-reputation loss if detected is $\rho_{\mathcal{A}} \cdot \Lambda \cdot p_d$ where $\rho_{\mathcal{A}}$ is the per-attestor reputation share of $\mathcal{A}$, Λ is the slash severity, and p_d is detection probability.

Bound (qualitative). The attestor set issues honestly when

$$\rho_{\mathcal{A}} \cdot \Lambda \cdot p_d > r_{\mathcal{T}} \cdot \gamma_{\mathcal{A}}^{-1}$$

where $\gamma_{\mathcal{A}} > 1$ is $\mathcal{A}$'s risk-aversion coefficient over reputation loss (analogous to the master-side γ in the Native Delegation v0.2 §5.5 inheritance theorem). For the standard PoUA reputation forward-revenue calibration (PoUA v0.9.2 §6.3 + §5.5.3 Lemma 1) and the recommended $\tau_{\text{burn}} \in [0.3, 0.5]$ range, the bound is satisfied for any $r_{\mathcal{T}}$ less than approximately 30 to 50 times the per-mint base fee at steady-state $b_{\sigma_{\text{mint}}}$.

What this means. Honest issuance is the dominant strategy when one excess-mint's market value is less than $\sim$ 30-50$\times$ the per-mint protocol fee. For an attestor set issuing a stablecoin with $b_{\sigma_{\text{mint}}} \sim 100$ uavow per mint and unit price ~ 1 AVOW per stablecoin unit, this margin is wide; the attestor set has no rational incentive to exceed the cap. For an attestor set issuing a very-high-unit-price token (e.g., a one-time-issued license NFT worth thousands of AVOW), the margin tightens; the §4.2 explicit cap-exceedance slash becomes the load-bearing defense, not the per-mint fee market.

Bound (quantitative, deferred). A precise parameterized bound requires devnet-observed values for $\rho_{\mathcal{A}}$ distribution across attestor sets, realized p_d from §A.3 detector calibration on real chain-graph data, and market-price distributions for representative tokens. Documented as a v0.3 mechanism-design refinement; the qualitative bound is sufficient to set the §4.2 cap-exceedance slash parameters today.

4.2 Cap-exceedance slash severity (still open, v0.3 work)

Tracked at ligate-chain#51. A new severity class for "schema-bound-token cap exceedance" needs to be added to the attestation slashing module. The §4.1 bound above suggests the severity should scale with `(realized_supply - stated_cap) / stated_cap`; specific severity-curve calibration awaits devnet data on realized cap-exceedance attempts (presumably zero or near-zero in honest operation; the parameter only fires under adversarial deviation).

Not resolved at v0.2.

4.3 Recall notice period calibration (resolved at v0.2)

v0.1 documented the recall mechanism with a placeholder 7-day notice period. v0.2 closes the calibration using a cross-product reference: the Time-Locked Attestations v0.2 §4.2 reveal-window mechanics give a clean analytical frame.

Specification. A `MsgRecall` issued by attestor set $\mathcal{A}$ for token type $\mathcal{T}$ does not execute immediately. Instead, the recall enters a `PENDING_RECALL` state at height H_{recall}, and the actual burn from holder balances fires at $H_{\text{recall}} + \text{notice\_period}$. During the notice window:

- Holders can transfer the token freely (the chain does not freeze balances during notice; freezing would create its own UX and legal issues that recall-with-notice avoids)
- An appeal can be filed via the §5.5.5 governance pathway (same machinery as PoUA reputation appeals); if the appeal succeeds, the pending recall is cancelled
- The recall itself is recorded as a `chain.token-recall-pending/v1` attestation at H_{recall}; the eventual burn is a separate `chain.token-burn/v1` attestation at $H_{\text{recall}} + \text{notice\_period}$. Auditability is symmetric.

Notice period calibration. Per-schema configurable, bounded by the protocol at a minimum of 24 hours and a maximum of 90 days. Recommended defaults by token category:

Token category	Notice period	Rationale
Regulated currency (e.g., stablecoin)	24 hours	Compliance flows need fast turn; holders are typically institutional with monitoring
Public-utility NFT (e.g., licenses)	7 days	Mirrors the PoUA §5.5.5 appeal window; balances institutional + retail holders
DAO governance token	Recall not enabled	DAO holdings are permanent by design
Privacy-sensitive credential	30 days	Holders need time to rotate underlying keys + migrate dependent state

Connection to time-locked attestations. The §3.3 validity state machine of Time-Locked Attestations v0.2 (`COMMITTED` → `REVEALED` / `EXPIRED` / `CLEANED-UP`) provides a clean parallel for recall lifecycle: `PENDING_RECALL` → `EXECUTED` / `APPEALED_CANCELLED` / `EXPIRED_UNAPPEALED`. Implementation can reuse the time-locked-attestations runtime extension when it ships; in the interim, schema-bound tokens declare their notice period at registration and the runtime tracks the state machine via the existing attestation-state-machine support.

4.4 Sub-quorum partial mint (still open, v0.3+ work)

v0.1 specified the same threshold k for both schema attestation and mint authorization. v0.2 carries this forward unchanged. The case for a stricter mint threshold ($k + 1$-of-$|\mathcal{A}|$) is real for high-value tokens but requires per-token-type policy specification; v0.3 should add a `mint_quorum_threshold` field to the token-type registration object with default = schema quorum, override = stricter. Tracked as a v0.3+ extension; not load-bearing for v0.2.

4.5 Sublicensing via meta-schemas (resolved at v0.2)

v0.1 framed this as open pending the Cross-Schema Composition primitive. With CSC v0.2 shipped, the resolution is direct.

Mechanism. A meta-schema σ_{meta} has an input-type-set $I_{\sigma_{\text{meta}}}$ (per CSC v0.2 §4.2) that requires references to a parent token's `chain.token-mint/v1` attestations. The meta-schema's attestor set $\mathcal{A}_{\text{meta}}$ is a *sub-quorum* of the parent token's attestor set $\mathcal{A}$ (in personnel; signed at registration by the parent quorum). When $\mathcal{A}_{\text{meta}}$ mints a derived token type $\mathcal{T}_{\text{derived}}$, the mint attestation references the parent token's most recent valid mint as its CSC input.

Type contract. The CSC v0.2 §4.3 type predicate $P_{\sigma_{\text{meta}}}$ verifies that:

1. The referenced parent-token mint attestation is in `VALID` state (CSC v0.2 §3.4 validity state machine);
2. The derived-token quantity is within the parent's authorized sublicensing budget (a `sublicense_cap` field on the parent's mint attestation's payload);
3. The sub-quorum signing $\mathcal{T}_{\text{derived}}$ matches a $\mathcal{A}_{\text{meta}}$-membership attestation pre-signed by $\mathcal{A}$.

Cascade semantics. Per CSC v0.2 §5.5 termination theorem, if the parent token's mint attestation is later slashed or revoked, the derived token's mint attestation cascades to `DEPENDENT-INVALID`. Holders of the derived token see the cascade through the read API; downstream consumers can choose strict-cascade behavior (burn derived balances automatically) or lazy-cascade behavior (report the invalidation but leave derived balances in place; the application decides recourse).

What this enables. A licensing-NFT issuer (e.g., a software-license attestor set) can authorize regional resellers as sub-quorums, each with their own mint authority for a derived token under a per-region `sublicense_cap`. The parent attestor set retains revocation power; the resellers operate independently within their cap. Without CSC v0.2's typed references and cascade machinery, this would be application-layer logic; with them, it is a runtime-primitive composition.

Open follow-up. Whether sublicensing can be recursive (sub-sub-quorums of sub-quorums) maps to CSC v0.2 §10.1 (recursive delegation deferred to v0.3). Not addressed here.

---

5. Comparison to existing patterns

Pattern	Mint authority	Issuance auditability	Quorum verification	Reputation tie-in
Standard ERC-20 (admin role)	Single address	State diff only	None	None
Safe / multisig wallet ERC-20	M-of-N approval contract	State diff only	Contract execution	None
Threshold-signature issuance (FROST-based)	Threshold of distributed keys	Signed txs on issuance chain	Bolted on per protocol	Varies by protocol
EAS revocable attestations	Schema resolver contract	Per-attestation contract call	Resolver-dependent	None
Schema-bound (Ligate)	**Attestor-set id**	**Mint = attestation under a system schema**	**Native chain primitive**	**PoUA reputation to attestor set + including validators**

The differentiators worth naming:

- **Threshold verification is in the consensus pipeline**, not in a contract. The chain natively understands what an attestor set is, what its threshold is, and how to verify a quorum signature. There is no separate primitive to deploy and audit.
- **Mint events are queryable as attestations** under a system schema. There is no separate token-issuance ledger; the audit infrastructure that indexes attestations is the audit infrastructure for token issuance.
- **Reputation flows through** to the same attestor set that authorized the mint. Fraudulent mints face the same economic floor (Lemma 1 in PoUA §5.5.3) as any other grinding behavior.

Neither Safe-style multisig nor EAS achieves any of these in a single integrated package. They each solve a slice. Schema-bound tokens are the integrated primitive.

---

6. Concrete use cases

Four use cases per ligate-chain#286. One worked through here in detail (use case B); three sketched.

6.1 (A) Regulated digital currency (the worked use case, v0.2)

A consortium of banks or a central bank issues a fiat-pegged digital currency on Ligate Chain. The issuer is an attestor set of the consortium's members (banks, central-bank divisions, an issuance-policy committee). Each member runs a validator and holds one signing share. The threshold k is set so that issuance requires a meaningful majority (e.g., 5-of-7, 7-of-10).

Construction. At consortium formation:

1. The consortium registers an attestor set $\mathcal{A}_{\text{stablecoin}}$ with the threshold k over the $|\mathcal{A}|$ member signing keys.
2. The consortium registers a schema-bound fungible token type $\mathcal{T}_{\text{stable}}$ with `mint_authority = AttestorSet($\mathcal{A}_{\text{stablecoin}}$)`, `recall_by_authority = true`, `recall_notice_period = 24 hours` (per §4.3 regulated-currency calibration), and a `supply_cap_per_epoch` parameter governing how fast the consortium can expand supply.
3. The consortium publishes a public reserves-attestation schema σ_{reserves} (off the schema-bound-tokens scope; lives in the audit-attestation family). Each mint of $\mathcal{T}_{\text{stable}}$ is accompanied by a corresponding σ_{reserves} attestation showing the underlying reserve increase.

Mint flow per issuance.

1. A consortium member's customer (a bank's institutional client) requests issuance of n stablecoin units against deposited reserves.
2. The bank submits the reserve evidence to the consortium's signing process (off-chain, the consortium runs its own quorum-collection infrastructure; this is application layer, not chain layer).
3. The consortium collects k signatures (the threshold quorum). The signed payload is `(token_id($\mathcal{T}_{\text{stable}}$), recipient_address, n, nonce, reserve_attestation_ref)`.
4. The submitter (typically the requesting bank, acting as the chain-side transaction relay) submits a `MsgMint` carrying the threshold-signed attestation. The chain validates the threshold quorum signature, the supply-cap-per-epoch budget, and the reserve-attestation reference

(CSC v0.2 typed reference check per §4.5 of this paper); if all three pass, the mint lands as a `chain.token-mint/v1` attestation.
5. Customer's stablecoin balance increases by n in the chain's runtime state.

Audit flow per regulator.

A regulator queries the chain for "all mints of $\mathcal{T}_{\text{stable}}$ in [date_range] by [bank_id]." This is one query against the attestation log filtered by schema σ_{mint}, token_id, time range, and submitter address. Returns the full audit trail with the threshold signatures, the timestamps, and the reserve-attestation references. The regulator cross-references against the consortium's reserve attestations under σ_{reserves} via the CSC v0.2 typed-reference graph; if any mint is missing a reserve attestation or references one that has since been invalidated, the cascade machinery (CSC v0.2 §5) flags the dependent mint as `DEPENDENT-INVALID`.

Recall flow (regulatory burn-on-court-order).

When a court order requires burning balances held by a specific address (e.g., sanctions enforcement, court-ordered restitution), the consortium executes a `MsgRecall`:

1. The threshold quorum signs a recall attestation under `chain.token-recall-pending/v1` for the affected balance.
2. The recall enters the 24-hour notice window per §4.3 calibration.
3. During the notice window, the affected holder can file an appeal via the §5.5.5 governance pathway (per PoUA v0.9.2 §5.5.5 appeal machinery). If the appeal succeeds before notice expires, the pending recall is cancelled.
4. If no appeal succeeds, the balance is burned at $H_{\text{recall}} + 24\text{h}$ via a `chain.token-burn/v1` attestation referencing the original recall.

Why the schema-bound construction beats the alternatives. Compared to issuing the stablecoin on an EVM chain via a multisig wallet contract:

- **Issuance audit**: the consortium's authorization is verifiable in the same place every regulator looks (the attestation log), not in a smart contract's storage layout.
- **Reserves linkage**: the CSC v0.2 typed-reference machinery makes mint-without-reserves a *chain-level* error, not an application-level audit gap.
- **Recall procedure**: the 24-hour notice window is enforced by the runtime; there's no smart-contract logic to audit for race conditions or holder-disempowerment.
- **Reputation feedback**: a member of the consortium who consistently signs off on mints without proper reserve-attestation references is subject to PoUA reputation slashing, which is more directly painful than the contract-layer governance procedures of typical multisig wallets.

Open product questions. The 24-hour notice period is the regulatory-currency default per §4.3; some jurisdictions may require longer (7 days for retail-facing recalls) or shorter (immediate for AML cases). The chain enforces a protocol-level bound between 24 hours and 90 days (§4.3); within that bound, the consortium configures the parameter at registration. v0.3+ may add per-recall override (e.g., "this specific recall qualifies for the 0-hour AML exception") with stricter justification requirements; out of scope for v0.2.

6.2 (B) AI-provenance content as NFTs (the worked use case)

Themisra v1 ships canonical AI-provenance schemas: `themisra.proof-of-prompt/v1`, `themisra.content-provenance/v1`. A natural extension is to allow content authors to mint a *unique, verifiable token* representing the AI-provenance receipt for a specific artifact. The mint authority is the same attestor set that signs the underlying provenance attestation.

Construction. When a content author submits a piece of AI-generated content for attestation, the Themisra attestor set signs a `themisra.content-provenance/v1` attestation. The author then mints an NFT under a schema-bound token type whose mint authority is *the same attestor set*. The NFT's payload includes the hash of the original content + the attestation id of the provenance record. The token is a 1-of-1 mint (NFT semantics) with the content hash as the implicit uniqueness key.

Why this is interesting. Compared to the existing pattern of "issue an NFT on an EVM chain referencing an off-chain attestation," the schema-bound construction has three advantages:

1. The NFT cannot exist without the provenance attestation existing first. The same attestor set authorizes both. There is no race condition where the NFT mints before the attestation, or persists after the attestation is invalidated.
2. The audit trail for the NFT and the audit trail for the provenance record are *the same audit trail*. A buyer verifying the NFT's authenticity verifies the provenance in one query.
3. If the provenance attestation is later shown to be fraudulent (e.g., the content was actually human-produced but mis-attested as AI), the recall mechanism can burn the NFT, and the §A.3 reputation feedback loop slashes the attestor set's reputation. The recall is a feature of the integrated trust model, not a hack bolted on.

v0.2 work. Specify the on-chain reference from the NFT to the provenance attestation (likely a `provenance_attestation_id: AttestationId` field in the NFT's payload), the recall conditions (when can the attestor set burn? automated trigger if the underlying provenance is invalidated, or only via governance?), and the marketplace-side queries (how does an NFT marketplace check that the linked provenance is still valid?).

Connection to chain#384 (Themisra Prompt Marketplace). The prompt-marketplace work uses similar machinery: an attestor set authorizes prompt-template publication, the template is sold with royalty terms, and each downstream use is itself an attestation. The schema-bound NFT primitive is the natural data type for the templates themselves; the marketplace then routes royalties via the per-schema fee mechanism documented in the Per-Schema Fees paper.

6.3 (C) DAO treasury tokens

Sketch only. A DAO issues governance tokens whose mint authority is an attestor set representing the DAO's elected representatives. Issuance events are queryable. Recall is *not* enabled (DAO members hold balances permanently). Threshold rotation matches the DAO's election cycle.

6.4 (D) Regulated licenses as NFTs

Sketch only. A regulator (e.g., a professional licensing board) issues licenses as NFTs whose mint authority is the regulator's attestor set. Issuance is auditable. Recall is enabled and routes through the regulator's existing revocation process. Each license NFT carries the licensee's identifier and the license terms in its payload.

---

7. Where this lives in the canon

Per ligate-research#84 §7, this is a **standalone research note** living under its own paper directory in the research repo. It is not absorbed into PoUA (which would expand PoUA's scope beyond consensus weighting) and not folded into Native Delegation (which has a different thematic focus on validator-attestor key separation).

The note cross-links to:

- PoUA v0.9.2 for the attestor-set + threshold-signature mechanics that this note builds on
- Per-Schema Fees v0.2 for the fee-market integration in §3.3 + §3.6
- Cross-Schema Composition v0.2 for the typed-reference primitive used in §4.5 sublicensing + §6.1 regulated-currency reserves linkage
- Time-Locked Attestations v0.2 for the validity-state-machine parallel used in §4.3 recall notice-period mechanics
- Native Delegation v0.2 for the risk-aversion-coefficient framing referenced in §4.1
- ligate-chain#286 for the engineering RFC
- ligate-chain#258 for the `$AVOW` economics
- ligate-chain#47 and #48 for the standard token primitives

Roadmap

- **v0.1 (2026-05-19)**: formal properties §3.1-§3.5 written; one use case (B, AI-provenance NFTs) worked through; comparison table populated; game-theoretic open questions listed.
- **v0.2 (this draft, 2026-05-25)**: §3.6 fee-market composition closed using Per-Schema Fees v0.2 mechanism; §4.1 attestor-set cap exceedance incentive analysis resolved qualitatively (§4.2 quantitative slash severity remains v0.3 work); §4.3 recall notice-period calibration closed with category-specific defaults; §4.5 sublicensing via meta-schemas closed using CSC v0.2 typed references; §6.1 regulated-currency use case expanded from sketch to full worked example. Cross-links updated for v0.2 of all dependent papers.
- **v0.3 (target Q3 2026, post-devnet observation)**: §4.2 cap-exceedance slash severity calibration once devnet data exists; §4.4 sub-quorum partial mint mechanism if design-partner demand validates it; per-unit-minted Pigouvian-tax economics analysis if ligate-chain#258 lands a tax-design decision.
- **v1.0 (post-mainnet)**: stable. Either folded into a "Token Primitives on Ligate" survey paper alongside #47, #48, or kept as a standalone reference for this specific primitive.

Out of scope for this note

- Engineering design (lives in ligate-chain#286)
- `$AVOW` economics around mint fees (lives in ligate-chain#258)
- Token contract code (lives in ligate-chain#47, #48, follow-up implementation issues)
- EVM-compatible ERC-20 wrapping (lives in `ligate-chain#52`)

End of working paper v0.2. Comments welcome to hello@ligate.io.